

Autorzy:

Gabriela Błaut
Michał Brzeziński
Aleksandra Gołek



Github projektu: <https://github.com/gabbla05/KittyProtocol>

KittyProtocol / Etap 1

1. Cel i zakres protokołu

★ **Cel (Do czego służy protokół?):**

Celem *KittyProtocol* jest umożliwienie **bezpiecznej, efemerycznej i błyskawicznej wymiany krótkich wiadomości tekstowych** w architekturze Klient–Serwer, przy jednoczesnym:

- braku przechowywania historii wiadomości na serwerze,
- braku przechowywania jakichkolwiek kluczy kryptograficznych na serwerze,
- pełnym szyfrowaniu end-to-end realizowanym wyłącznie po stronie klientów,
- odporności na podsłuch, manipulację i ataki typu Man-in-the-Middle.

Protokół zakłada, że para klientów ustala wspólny sekret (Pre-Shared Secret, PSS) poza systemem (out-of-band), np. poprzez spotkanie fizyczne, telefon, kod QR lub inny bezpieczny kanał.

★ **Rozwiązywane problemy:**

Protokół stanowi lekką alternatywę dla obciążających, tradycyjnych komunikatorów, skupiając się na maksymalnej prywatności i szybkości. Rozwiązuje dwa kluczowe problemy sieciowe:

- ★ **Wysokie opóźnienia:** Dzięki oparciu o warstwę transportową QUIC, *KittlyProtocol* znacząco minimalizuje czas nawiązywania połączeń.
- ★ **Brak natywnego wsparcia dla mobilności:** Protokół rozwiązuje problem zrywania sesji przy częstych zmianach sieci. Zapewnia ciągłość sesji i utrzymanie komunikacji bez konieczności ponownego logowania użytkownika.

Oraz problemy związane z bezpieczeństwem:

- ★ **Atak MitM:** Możliwość podszywania się pod użytkownika w trakcie komunikacji
- ★ **Poufność:** Żadne urządzenie poza nadawcą i odbiorcą nie jest w stanie odczytać wiadomości
- ★ **Integralność:** Dzięki użyciu protokołu QUIC, który wykorzystuje TLS 1.3 uniemożliwia naruszenie spójności przesyłanych ramek.

★ Model komunikacji:

Komunikacja odbywa się w modelu **Klient-Serwer (C/S)** z zastosowaniem pełnego szyfrowania End-to-End (E2EE). Serwer centralny (Hub) pełni rolę routera wiadomości oraz zarządcy tożsamości. Taka architektura rozwiązuje całkowicie problem widoczności węzłów w tablicach NAT. Ponieważ serwer jedynie przekazuje zaszyfrowane asymetrycznie ładunki (payload), nie ma możliwości odczytania treści konwersacji.

2. Założenia techniczne

★ Transport:

Protokół wykorzystuje warstwę transportową QUIC. Wybraliśmy QUIC, ponieważ łączy on szybkość protokołu UDP z niezawodnością TCP. Kluczowym argumentem był mechanizm 0-RTT Connection Establishment, który znacząco skraca czas nawiązywania połączenia. Co prawda 0-RTT jest podatne na replay attacks, a QUIC/TLS 1.3 *nie gwarantuje*, że dane wysłane w 0-RTT są odporne na powtórzenie. Natomiast jest to rozwiązane unikalnym *msg_id*. Dodatkowo, dzięki zastosowaniu jednego strumienia na jedno połączenie zapewniamy kolejność przychodzących wiadomości. Wybór ten zapewnia również natywne szyfrowanie TLS 1.3 oraz stabilność połączenia przy zmianie sieci (np. Wi-Fi na LTE) dzięki wykorzystaniu Connection ID.

★ Kodowanie wiadomości:

Jako format kodowania wybraliśmy tekstowy format JSON. Łączy on czytelność dla człowieka z wysoką wydajnością parsowania. Wybór ten ułatwia debugowanie (struktura ramek jest łatwo widoczna np. w Wiresharku) oraz eliminuje niskopoziomowe problemy z kolejnością bajtów (Endianness) charakterystyczne dla protokołów binarnych. Elastyczność JSONa pozwala na łatwe rozszerzanie protokołu o nowe pola w przyszłości bez naruszania kompatybilności wstecznej.

★ Założenia dotyczące niezawodności:

Zadania związane z niezawodnością zostały wyraźnie podzielone między warstwę transportową a aplikacyjną.

★ Co zapewnia warstwa transportowa (QUIC):

Gwarantuje integralność dostarczanych danych, zachowanie odpowiedniej kolejności pakietów w ramach poszczególnych strumieni oraz automatyczną retransmisję zagubionych fragmentów.

★ Co zapewnia protokół aplikacyjny (*KittyProtocol*):

■ Potwierdzenia logiczne (Application-level ACK):

Komunikat **MEOW_OK** jest wysyłany przez odbiorcę dopiero po pełnej, poprawnej walidacji struktury JSON przez aplikację. Oznacza to, że wiadomość została nie tylko dostarczona (ACK poziomu warstwy transportowej z QUIC), ale też zrozumiana i zaakceptowana.

■ Kontrola limitów (Rate Limiting):

Ograniczamy liczbę komunikatów do 10 na sekundę dla jednego użytkownika (oraz max 100 wiadomości na minutę z jednego IP), aby zapobiegać nadużyciom typu spam/DoS. Testowanie: e.g. wysłanie 1000 zapytań do węzła/serwera pomocniczego.

■ Spójność doręczeń (Delivery ACK):

Delivery ACK: w komunikacji C/S odbiorca wyśle potwierdzenie odebrania wiadomości (Delivered), oznaczające, że dane zostały poprawnie odebrane na poziomie transportu. Nie gwarantuje to ich przetworzenia przez aplikację (obsługa w warstwie aplikacji).

- **Timeouty aplikacyjne:**

Wprowadzamy rygorystyczny, 20-sekundowy limit na zakończenie procesu uwierzytelnienia (**AUTH**). Brak odpowiedzi skutkuje błędem ERR_03 (Timeout) i jednostronnym zamknięciem sesji.

3. Struktura komunikatów (Formatki)

★ **Typy wiadomości:**

Zdefiniowano następujące główne typy komunikatów sterujących i wymiany danych (wartości dla pola type):

- ★ **HELLO** – **Inicjalizacja sesji.** Klient rozpoczyna sesję aplikacyjną z Hubem, wysyłając komunikat HELLO zawierający wyłącznie identyfikator wiadomości **msg_id**. Na tym etapie nie są przesyłane żadne klucze kryptograficzne – służy to jedynie do zainicjowania sesji i przygotowania do procesu uwierzytelnienia.
- ★ **AUTH** – uwierzytelnianie użytkownika w Hubie.
- ★ **GET_STATUS** – **Zapytanie klienta do Huba o status docelowego odbiorcy.** Ramka służy do sprawdzenia, czy wskazany użytkownik jest aktualnie online i może przyjmować wiadomości.
- ★ **STATUS_RES** – **Odpowiedź Huba na zapytanie GET_KEY.** Zawiera informację o tym, czy odbiorca jest online.
- ★ **DATA** – przesyłanie właściwego ładunku (tekstu/obrazków ASCII) między użytkownikami.
- ★ **MEOW_OK** – uniwersalne potwierdzenie logiczne na poziomie aplikacji (Application-level ACK).
- ★ **ERROR** – komunikowanie błędów walidacji, uwierzytelnienia lub timeoutów.
- ★ **PING** – wiadomości Keep-alive podtrzymujące sesję, wysyłane do serwera.
- ★ **BYE** – poprawne i celowe zakończenie sesji.

★ **Pola wiadomości (wymagane i opcjonalne):**

Każda ramka jest poprawnym obiektem JSON.

Pola wymagane (w każdym komunikacie):

- **type** (String) – określa rodzaj wiadomości (np. "DATA").

- **msg_id** (timestamp) - unikalny identyfikator wiadomości w ramach sesji. Chroni przed duplikatami na poziomie aplikacji i służy do mapowania potwierdzeń (odbiorca odsyłając **MEOW_OK** załącza ten sam **msg_id**). Zamiast tradycyjnego **msg_id** jako inkrementowanego integera, identyfikatorem wiadomości w sesji staje się bezpośrednio **timestamp** (np. **290326213840**). Warstwa transportowa QUIC zajmuje się retransmisją zgubionych pakietów.

Pola specyficzne / opcjonalne (zależne od typu):

- **payload** (String) – właściwa treść wiadomości, wymagana dla **DATA**.
- **mac** (String) - HMAC wiadomości
- **user, pass** (String) – dane logowania, wymagane wyłącznie dla **AUTH**.
- **target** (String) – identyfikator odbiorcy, wymagany w ramce **DATA** oraz **GET_STATUS**.
- **code, desc** (String) – kod i opis błędu, występują tylko w komunikatach **ERROR**

★ Format (przykład ogólnej ramki):

Wszystkie komunikaty opierają się na jednej spójnej strukturze podstawowej. Przykład pełnej ramki z danymi:

JSON

```
{
  "type": "DATA",
  "msg_id": 1709048120311,
  "target": "bob",
  "payload": "BASE64(szyfrogram)",
  "mac": "BASE64(HMAC_SHA256(payload || msg_id || target))"
}
```

Uwaga architektoniczna:

HMAC liczone jest osobnym, end-to-end kluczem między klientami (nieznanym serwerowi). Zatem HMAC może służyć jako dodatkowa warstwa E2E-integralności / wykrywania MitM *ponad* transportem.

★ Co musi zostać w ramce?

- "type": "DATA" - Protokół QUIC operuje na strumieniach bajtów i nie narzuca semantyki przesyłanych danych. Warstwa aplikacyjna musi więc jednoznacznie określać typ komunikatu, aby możliwa była jego poprawna interpretacja (np. wiadomość tekstowa, PING, BYE).
- "msg_id": timestamp - Mimo że QUIC zapewnia niezawodność transmisji oraz zachowanie kolejności danych w obrębie pojedynczego strumienia, identyfikator wiadomości jest niezbędny na poziomie logiki aplikacyjnej. Umożliwia on:
 - ❖ jednoznaczne powiązanie komunikatu z odpowiadającym mu potwierdzeniem (**MEOW_OK**),
 - ❖ obsługę ewentualnych retransmisji na poziomie aplikacji,
 - ❖ poprawne odwzorowanie stanu wiadomości w interfejsie użytkownika (np. oznaczenie „Dostarczono”).

Dodatkowo sam w sobie "Timestamp" określa czas utworzenia wiadomości i pełni istotną rolę w logice aplikacyjnej, w szczególności:

- ❖ umożliwia wykrywanie i odrzucanie przestarzałych lub zreplayowanych komunikatów,
 - ❖ wspiera ewentualne porządkowanie wiadomości niezależnie od mechanizmów transportowych,
 - ❖ ułatwia debugowanie oraz analizę zdarzeń w systemie rozproszonym.
- "payload": "Meow! ASCII art." - Zawiera właściwe dane aplikacyjne przesyłane pomiędzy peerami. QUIC nie ingeruje w strukturę ani interpretację tego pola.

★ Walidacja (co uznajemy za błąd formatu):

Aplikacja rygorystycznie sprawdza każdą przychodzącą ramkę. Za błąd formatu (skutkujący natychmiastowym odrzuceniem wiadomości i odesłaniem komunikatu **ERROR** z kodem ERR_02) uznaje się:

- Błąd parsowania: Otrzymany ciąg znaków nie jest poprawnym dokumentem JSON (np. brakujące klamry, cudzysłowy).
- Brak pól bazowych: W obiekcie JSON brakuje klucza **type** lub **msg_id**.
- Nieznany typ: Klucz **type** zawiera wartość spoza dozwolonej listy.
- Niezgodność typów zmiennych: Np. pole **msg_id** zostało przesłane jako ciąg znaków (String) zamiast liczby całkowitej (Integer).

4. Model stanów i przebieg komunikacji

★ Opis sesji (Przebieg komunikacji):

Cykl życia każdej sesji w protokole *KittyProtocol* dzieli się na 4 rygorystyczne fazy:

Faza 1 – Handshake (HELLO)

Klient → Hub:

```
{ "type": "HELLO", "msg_id": 1709048120300 }
```

Hub → Klient:

```
{ "type": "MEOW_OK", "msg_id": 1709048120301, "status":  
  "Ready for auth" }
```

Faza 2 – Logowanie (AUTH)

Klient → Hub:

```
{ "type": "AUTH", "msg_id": 1709048120310, "user":  
"alice", "pass": "secret" }
```

Hub → Klient:

```
{ "type": "MEOW_OK", "msg_id": 1709048120311, "status":  
"Logged in" }
```

Serwer przechowuje **wyłącznie login + hasło (hash)**.

Faza 3 – Sprawdzenie statusu odbiorcy

Klient → Hub:

```
{ "type": "GET_STATUS", "msg_id": 1709048120320, "target":  
"bob" }
```

Hub → Klient:

```
{ "type": "STATUS_RES", "msg_id": 1709048120321, "target":  
"bob", "status": "online" }
```

Faza 4 – Wymiana danych (E2EE)

Ustalanie wspólnego sekretu (OOB - out of band)

Alice i Bob ustalają poza systemem:

K_{AB} = 32-bajtowy wspólny sekret

Z niego wyprowadzają:

$K_{enc} = \text{KDF}(K_{AB}, \text{"encryption"})$

$K_{mac} = \text{KDF}(K_{AB}, \text{"authentication"})$

Wysyłanie wiadomości

Alice szyfruje plaintext:

```
cipher = AEAD_Encrypt(K_enc, nonce=msg_id, plaintext)
```

```
mac = HMAC_SHA256(K_mac, cipher || msg_id || target)
```

Alice → Hub:

```
{  "type": "DATA", "msg_id": 1709048120330, "target":  
  "bob",  
  
  "payload": "BASE64(cipher)", "mac": "BASE64(mac)"}
```

Hub → Bob:

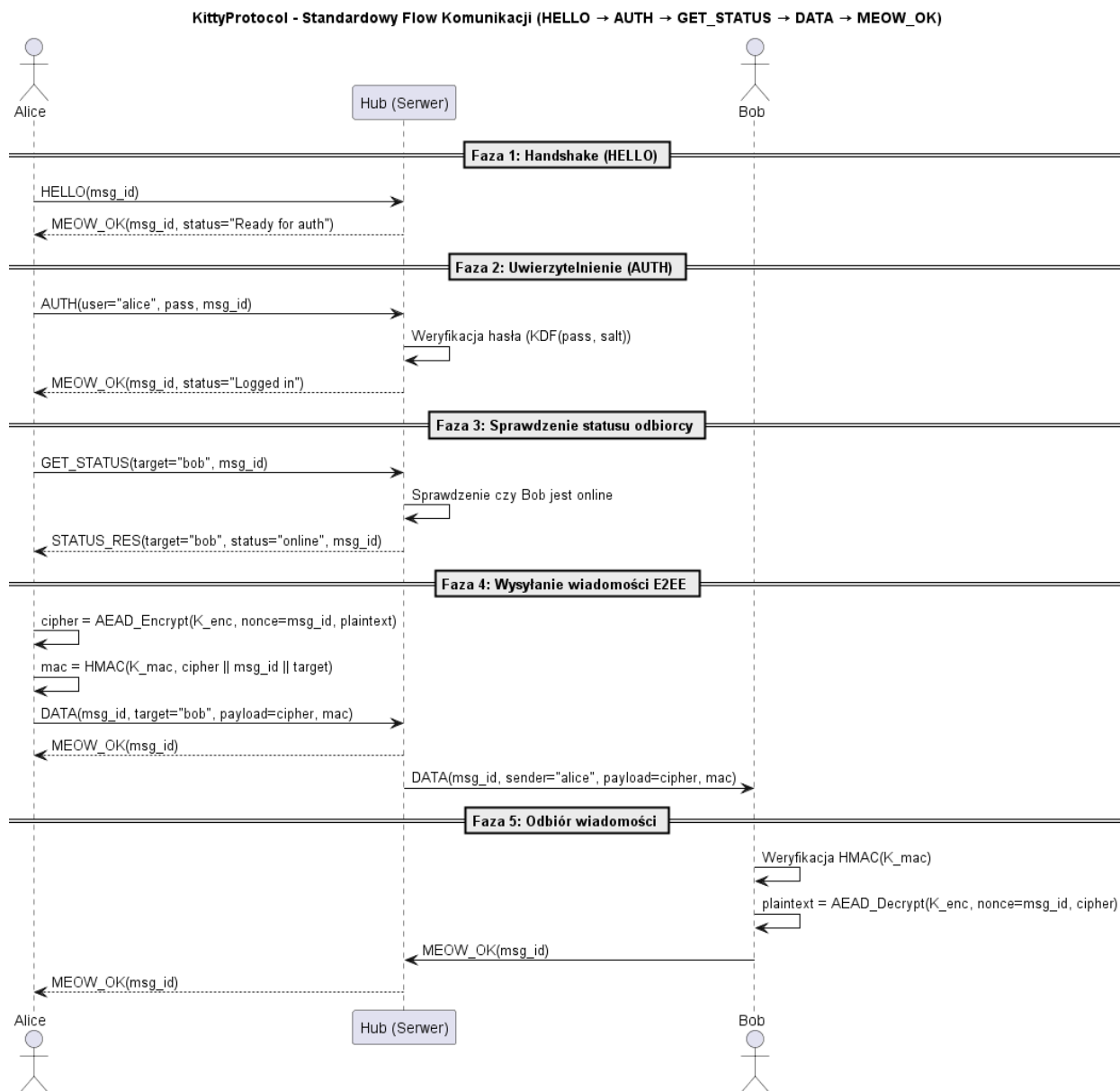
```
{  "type": "DATA", "msg_id": 1709048120330, "sender":  
  "alice",  
  
  "payload": "BASE64(cipher)", "mac": "BASE64(mac)"}
```

Bob:

- weryfikuje HMAC,
- odszyfrowuje wiadomość,
- odsyła MEOW_OK.

★ Przykładowy diagram sekwencji:

Schemat obrazuje prawidłowy przepływ wiadomości od nawiązania połączenia po wymianę danych.



★ Mechanizmy podtrzymania i niezawodności:

- **Keep-alive:** Aby zapobiec zamykaniu nieaktywnych strumieni QUIC przez urządzenia sieciowe (NAT/Firewall), wprowadzono zasadę wysyłania komunikatu PING przez klienta co 30 sekund.
- **Timeouty:** Timeout uwierzytelnienia: Wprowadzono rygorystyczny limit 20 sekund na poprawne przesłanie komunikatu AUTH od momentu wysłania HELLO. Przekroczenie tego czasu skutkuje błędem ERR_03 i przerwaniem połączenia .
- **Timeout sesji:** Jeśli odbiorca nie otrzyma od nadawcy żadnych danych (komunikatów DATA lub PING) w ciągu 60 sekund, następuje jednostronne zamknięcie sesji (uznanie hosta za offline).
- **Retransmisja / Retry:** Na poziomie warstwy 4 (QUIC) retransmisja zagubionych pakietów realizowana jest w pełni automatycznie i przezroczysto dla aplikacji.

Na poziomie aplikacyjnym: jeśli nadawca wyśle komunikat **DATA** i w ciągu 5 sekund nie otrzyma od serwera zwrotnego **MEOW_OK** (Application-level ACK), aplikacja **nie ponawia** wysyłki w ciemno (ponieważ za retransmisję zagubionych pakietów odpowiada natywnie QUIC). Zamiast tego wiadomość otrzymuje w interfejsie użytkownika status "Niedostarczona/Timeout", sugerując utratę połączenia ze strony drugiego hosta.

5. Bezpieczeństwo

★ Poufność i Integralność transportu:

Warstwa transportowa QUIC (z wymuszonym TLS 1.3) zapewnia pełną poufność i integralność wszystkich przesyłanych danych - zarówno struktury JSON, jak i zaszyfrowanego payloadu. QUIC wykorzystuje mechanizmy AEAD (Authenticated Encryption with Associated Data), które gwarantują:

- wykrycie każdej modyfikacji danych w locie (tampering),
- ochronę przed podsłuchem,
- ochronę przed klasycznymi atakami typu Man-in-the-Middle.

Aplikacja nie musi implementować dodatkowych HMAC ani sum kontrolnych — QUIC zapewnia to natywnie.

★ **Poufność danych (E2EE):** *KittyProtocol* stosuje szyfrowanie end-to-end na poziomie aplikacji.

Mechanizm działa następująco:

1. Klienci uzgadniają wspólny sekret (np. klucz prywatny).
2. Nadawca szyfruje wiadomość **za pomocą wspólnego sekretu**.
3. Hub jedynie przekazuje zaszyfrowane ramki **DATA** – nie ma dostępu do kluczy prywatnych klientów.

Dzięki temu:

- Hub nie może odczytać treści wiadomości,
- przejęcie serwera nie ujawnia historii rozmów a cała treść jest widoczna wyłącznie dla nadawcy i odbiorcy.
- Nawet Hub nie ma możliwości podszycia się pod jakiegokolwiek z urządzeń.

★ Uwierzytelnienie

Proces logowania w *KittyProtocol* polega na potwierdzeniu tożsamości użytkownika na podstawie loginu i hasła.

1. Klient przesyła do Huba ramkę **AUTH** zawierającą:
 - **user** – nazwę użytkownika,
 - **pass** – hasło w postaci plaintext wewnątrz szyfrowanego kanału QUIC/TLS 1.3.
2. QUIC/TLS 1.3 zapewnia pełną poufność i integralność transportu, dlatego hasło nigdy nie jest przesyłane w formie jawnej poza bezpiecznym tunelem.
3. Po stronie serwera hasło jest przetwarzane następująco:
 - serwer pobiera z bazy **password_hash** oraz **salt**,
 - oblicza **KDF(pass, salt)** (np. Argon2 / bcrypt),
 - porównuje wynik z zapisanym hashem.
4. Jeśli wartości są zgodne, serwer uznaje użytkownika za uwierzytelnionego i zwraca **MEOW_OK**.

Przykładowa struktura tabeli w bazie danych:

```
CREATE TABLE users (  
  
    id SERIAL PRIMARY KEY,  
  
    username VARCHAR(50) UNIQUE NOT NULL,  
  
    password_hash VARCHAR(255) NOT NULL,  
    -- Haszowane np. bcryptem/rsa!  
  
    is_online BOOLEAN DEFAULT FALSE,  
    -- Flaga st. obecności (opcjonalne, bo Hub może to trzymać w  
    RAM)  
  
);
```

★ Ochrona przed atakami Replay (Powtórzenia):

1. Ochrona na poziomie w. transportowej (TLS 1.3)

TLS 1.3 chroni przed powtórzeniami pakietów w ramach sesji QUIC.

2. Ochrona na poziomie w. aplikacji (msg_id + timestamp)

Każda wiadomość zawiera: **msg_id** – unikalny identyfikator (timestamp). Klient utrzymuje w RAM listę ostatnio przetworzonych **msg_id**, stąd jeśli pojawi się duplikat to ramka jest cicho odrzucana.

To zabezpiecza przed:

- ★ powtórzeniami 0-RTT,
- ★ wstrzyknięciem starych ramek,
- ★ atakami replay po stronie aplikacji.

★ Model zagrożeń:

KittyProtocol zakłada działanie w niezaufanym środowisku sieciowym (np. publiczne sieci Wi-Fi).

- **Podśluch, Man-in-the-Middle (MitM) i Tampering:**

Całkowicie mitygowane natywnie przez QUIC/TLS 1.3 oraz E2EE

- **Wyczerpanie zasobów (Ataki DoS / Spam):**

Zagrożenia minimalizowane poprzez:

- **Rate Limiting** po stronie Huba:
 - max 10 komunikatów/s na użytkownika,
 - max 100 komunikatów/min z jednego IP.
- rygorystyczną walidację ramek JSON,
- natychmiastowe zamykanie sesji przy naruszeniu protokołu.

- **Kompromitacja centralnego serwera (Hub'a):**

Nawet w przypadku przejęcia Huba treść wiadomości pozostaje niejawna (E2EE).

Atakujący może jedynie:

- zakłócać routing,
- blokować użytkowników,
- manipulować sygnalizacją.

Nie może:

- odszyfrować wiadomości,
- odczytać historii rozmów,
- podsłuchać komunikacji.

6. Obsługa błędów i awarii połączenia

Błędy w *KittyProtocol* są komunikowane asynchronicznie za pomocą ramek typu ERROR, co pozwala na natychmiastową reakcję aplikacji.

★ Kody błędów

Na podstawie dokumentu można wyróżnić następujące kody:

Kod	Znaczenie	Kiedy występuje	Zalecana reakcja
ERR_01	Protocol Violation	Np. Wysłanie DATA przed AUTH	Zakończyć sesję, rozpocząć od HELLO
ERR_02	Format Error	Niepoprawny JSON, brak type/msg_id , złe typy	Poprawić ramkę; po wielu błędach zakończyć sesję
ERR_03	Authorization Timeout	Brak AUTH w ciągu 20 s od HELLO	Wyświetlić timeout, ponowić logowanie
ERR_04	Authentication Failed	Nieprawidłowe dane logowania	Poprosić o poprawne dane
ERR_05	Session error	Niezdefiniowany błąd sesji	Wymusić ponowne uwierzytelnienie i próba ponownego nawiązania połączenia
ERR_06	Replay Detected	Powtórne użycie msg_id	Odrzucić ramkę, opcjonalnie zamknąć sesję
ERR_07	Rate Limit Exceeded	>10 komunikatów/s lub >100/min z IP	Zastosować backoff, odczekać
ERR_08	Delivery Failed – Recipient Offline	Odbiorca nie jest online	Poinformować nadawcę, brak kolejowania
ERR_09	Session Timeout (Idle)	Brak aktywności przez 60 s	Wznović połączenie lub wykonać handshake

ERR_10	Resource Exhaustion	Przeciążenie Huba	Retry z losowym jitterem
ERR_11	Internal Server Error	Błąd Huba (np. DB offline)	Retry po krótkim czasie
ERR_12	Version Mismatch	Nieobsługiwana wersja protokołu	Zaktualizować klienta
ERR_13	Payload Too Large	Payload > 2048 bajtów	Uciąć tekst i wysłać ponownie
ERR_14	Not Authorized	Brak uprawnień do akcji	Przerwać operację
ERR_15	Unknown Target	GET_KEY dla nieistniejącego użytkownika	Zakończyć próbę wysyłki

★ Zachowanie po błędach składni/protokołu:

- Każda ramka JSON przechodzi pełną walidację strukturalną.
- W przypadku błędu formatowego odbiorca natychmiast odsyła **ERROR** (**ERR_02**).
- Powtarzające się błędy lub naruszenia protokołu skutkują jednostronnym zamknięciem sesji.

★ Timeouty połączenia:

- Timeout uwierzytelnienia:

Proces **AUTH** musi zakończyć się w ciągu **20 sekund** od **HELLO**.

Po przekroczeniu limitu:

- Hub zwraca **ERR_03**,
- sesja jest zamykana,
- zasoby są zwalniane.

- **Timeout bezczynności:**

Hub monitoruje aktywność klienta.

Jeśli przez 60 sekund nie nadejdzie żaden **DATA** ani **PING** :

- sesja jest uznawana za zerwaną,
- Hub zamyka połączenie QUIC,
- klient musi wykonać ponowny handshake.

Timeouty pomagają ograniczyć ryzyko ataków typu replay oraz DoS poprzez automatyczne zamykanie nieaktywnych połączeń. Monitorowanie bezczynności po stronie serwera gwarantuje prywatność danych – serwer nie ma wglądu w jawną treść wymienianych wiadomości **DATA**.

★ **Utrata połączenia w trakcie sesji**

QUIC zapewnia:

- odporność na krótkie przerwy,
- migrację między sieciami (Connection ID),
- automatyczne retransmisje.

W przypadku trwałej utraty połączenia:

- klient wykonuje ponowny handshake,
- może wznowić sesję,
- aplikacja może wykorzystać ostatni **msg_id** do oznaczenia wiadomości jako niedostarczonych.

★ **Duplikaty i niekompletne wiadomości**

- **Ochrona przed duplikatami:**

- unikalny **msg_id**,
- pamięć ostatnich identyfikatorów po stronie klienta. (np. w RAM)

Duplikaty są cicho odrzucane.

★ Niekompletne wiadomości

- QUIC gwarantuje integralność strumienia.
- Wadliwa struktura JSON będzie skutkować błędem **ERR_02**.

★ Limity i ochrona przed nadużyciami

- Rate Limiting i Ochrona IP:

- max **1 komunikatów/s** na użytkownika,
- max **100 komunikatów/min** z jednego IP.

- Rozmiar komunikatu:

- maksymalny rozmiar ramki JSON: **4 KB**,
- maksymalny rozmiar payloadu po zaszyfrowaniu: **2048 bajtów**.

- Ucinanie (Truncation na poziomie aplikacji klienta - odbiorcy):

Jeśli użytkownik spróbuje wysłać wiadomość przekraczającą 2kB, aplikacja kliencka natychmiast ucina nadmiarowe znaki na poziomie interfejsu/logiki (jeszcze przed zaszyfrowaniem) i wysyła w sieć ramkę DATA mieszczącą się w limicie. Zapobiega to błędom i odrzuceniem po stronie Huba.

★ Format ramki błędu

W *KittyProtocol* błędy są komunikowane poprzez ramki typu **ERROR**, zawierające pola:

- **type**: "ERROR",
- **msg_id**,
- **code** – kod błędu/krótki opis typu błędu,
- **desc** – opis błędu

7. Scenariusze (Przykłady ramek)

Scenariusz 1: Poprawna sesja i wymiana wiadomości (E2EE przez Hub)

Standardowy przepływ od nawiązania połączenia z Hubem, został przedstawiony wcześniej w punkcie numer 4: **Model stanów i przebieg komunikacji**

Scenariusz 2: Błąd walidacji formatu (ERR_02)

Klient wysłał niepoprawny JSON (brak `type`, zły typ `msg_id`).

Klient → Hub

```
{  
  "msg_id": 1709048120330,  
  "payload": "Zaszyfrowany tekst..."  
}
```

Hub → Klient

```
{  
  "type": "ERROR",  
  "msg_id": "1709048120335",  
  "code": "ERR_02",  
  "desc": "Brak wymaganego pola type lub niezgodność typów"  
}
```

Scenariusz 3: Timeout procesu uwierzytelnienia (ERR_03)

Klient → Hub

```
{  
  "type": "HELLO",  
  "msg_id": "1709048120500",  
}
```

Hub → Klient

```
{  
  "type": "MEOW_OK",  
  "msg_id": "1709048120505"  
}
```

(Klient nie wysyła AUTH przez 20 sekund)

Hub → Klient

```
{  
  "type": "ERROR",  
  "msg_id": "1709048120500",  
  "code": "ERR_03",  
  "desc": "Authorization timeout reached"  
}
```

Scenariusz 4: Nieudane logowanie (ERR_04)

Klient → Hub

```
{  
  
  "type": "AUTH",  
  
  "msg_id": "1709048120312",  
  
  "user": "hacker",  
  
  "pass": "123"  
}
```

Hub → Klient

```
{  
  
  "type": "ERROR",  
  
  "msg_id": "1709048120318",  
  
  "code": "ERR_04",  
  
  "desc": "Authentication failed"  
}
```

Scenariusz 5: Odbiorca offline (ERR_15)

Klient → Hub

```
{  
  
  "type": "DATA",  
  
  "msg_id": "1709048120400",  
  
  "target": "bob",  
  
  "payload": "R3g4... [szyfrogram]"  
}
```

Hub → Klient

```
{  
  
  "type": "ERROR",  
  
  "msg_id": "1709048120406-A6",  
  
  "code": "ERR_15",  
  
  "desc": "Receiver offline"  
}
```

Scenariusz 6: Wiadomość dłuższa niż 2 kB

Aplikacja kliencka wykrywa przekroczenie limitu **2048 bajtów** i ucina tekst lokalnie.

Klient → Hub

```
{  
  
  "type": "DATA",  
  
  "msg_id": "1709048120500",  
  
  "target": "bob",  
  
  "payload": "R3g4... [szyfrogram przyciętej wiadomości]"  
}
```

Hub → Klient

```
{  
  
  "type": "MEOW_OK",  
  
  "msg_id": "1709048120508"  
}
```